《人工智能》Project 2：强化学习

- 课程号：CS50020；时间：2026.11

项目介绍

本课程项目分为课上实验和课下作业两部分：课上实验部分将实现值迭代和 Q-Learning 相关的核心代码；课下作业包括了基于核心代码的扩展实验和需要手写完成的作业题。本次课程项目会首先在网格世界（GridWorld）中测试你的智能体，然后将其应用于一个勇者斗恶龙游戏（Hero vs Dragons）中。

重要时间点

内容	开始时间	截止时间	提交方式	提交内容
课前热身	11.9	11.12 12:00	Elearning 提交	热身题答案
课上实验	11.12 13:30	11.12 16:10	每组当堂验收	讲解算法及结果
课后作业	11.12	11.22 23:59	Elearning 提交	报告 + 代码

作业要求

1. 1-2 人组队完成；
2. 发布和提交平台：项目的发布和提交都通过 elearning 平台完成；
3. 课后作业提交要求：
   - 每组提交一份 **PDF 报告**，报告须包含以下内容：
     (a) **课上实验回顾**：回顾课上实现的核心算法和代码，说明实现逻辑，并在报告中展示相应的关键代码；即使课上已验收，也必须在报告中回顾。
     (b) **课下扩展实验**：说明完成情况和实现逻辑，并在报告中展示相应的关键代码。
     (c) **自动评分结果**：附上能清楚显示最终各题得分和总分的自动评分结果截图；不能仅用文字自行陈述得分。
     (d) **手写作业题**：给出详细过程，可以手写后扫描或拍照，也可以电子版输入。
     (e) **组员信息与分工**：写明具体人员信息和各自分工。
     报告命名格式为任意一名同学的 学号-姓名-PJ2 报告.pdf；
   - 每组提交一份**代码附件压缩包**，只需将完成后的 `Agents/Coursework` 文件夹打包并上传，保留文件夹内的全部代码文件，无需提交其他实验文件。压缩包命名格式为与报告命名相同的同学的 学号-姓名-PJ2 代码.zip。**注意，请不要上传所有文件。**
4. 课程项目均需要使用 Python 编程，请提前安装好 Python 环境（推荐使用 Anaconda 或 uv）和集成开发环境（VS Code、PyCharm、Spyder 等）；
5. 课程项目的代码实现与手写作业题均**严禁抄袭**，相似度过高的作业提交将记为无效成绩；
6. 评分设置：课上核心代码实现当堂验收（30%）+ 课下作业报告提交（70%）。课上实验逾期未能验收时，仅验收最终报告内的课上实验介绍，并最多获得 $30\% \times 60\% = 18\%$ 的课上分数。

运行环境

请在 `PJ2-code-remake` 目录下执行本文命令。安装 Python 3 环境中的依赖：

```
python -m pip install -r requirements.txt
```

安装 PyTorch（可选）

我们不强制要求安装 PyTorch。Q7 的经验回放和探索率调度类可以在不安装 PyTorch 的环境中导入和实现；如果希望运行 Q7.3 的神经网络训练或加载模型测试，则需要安装 PyTorch。

安装时请先激活用于本项目的 Python 环境，再根据硬件选择以下一种方式。安装和运行训练时应使用同一个 Python 环境。

CPU 版本：不需要 NVIDIA 显卡，可在普通 CPU 上训练。

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
```

NVIDIA GPU 版本：需要支持 CUDA 的 NVIDIA 显卡以及兼容的显卡驱动。先运行 `nvidia-smi` 查看显卡和驱动信息，然后访问 PyTorch 官方安装页面，选择操作系统、Pip、Python 和适合机器的 CUDA 版本，执行页面生成的命令。例如，选择 CUDA 12.8 时可使用：

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cu128
```

实验介绍

在这个项目中，需要实现值迭代和 Q-Learning 相关的核心代码。

与之前的项目一样，本项目包含一个自动评分工具，可以用它来进行评分。运行以下命令，为所有问题评分：

```
python autograder.py
```

通过以下命令只运行特定问题的评分，例如 q2：

```
python autograder.py -q q2
```

参数 `-q` 控制待评估的题目。

当前自动评分覆盖 Q1–Q7，满分为 **25 分**；手写 Q8 另行评阅，不计入这 25 分。

在实现过程中，只需要关注其中的几个文件，可以忽视其他的支持文件：

需要编辑的文件

文件	说明
`Agents/Coursework/value_iteration_agents.py`	价值迭代的智能体，用来解决 MDPs。
`Agents/Coursework/qlearning_agents.py`	Q-learning 的智能体。
`Agents/Coursework/analysis.py`	用来提交 Q4 的答案。
`Agents/Coursework/feature_extractors.py`	Q6 特征提取器设计的入口。
`Agents/Coursework/dqn_utils.py`	Q7 的经验回放与探索率调度实现。

可以参考的文件

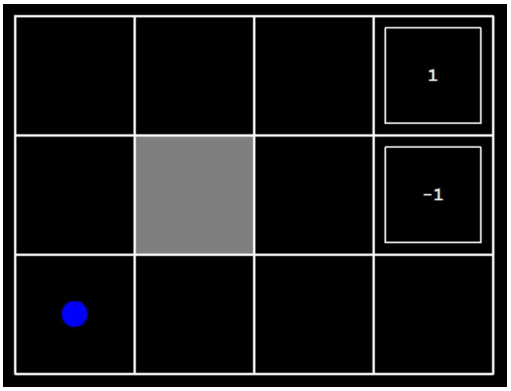
文件	说明
`Game/Core/mdp.py`	定义了 MDPs 的有关函数。
`Agents/Runtime/learning_agents.py`	定义了 `ValueEstimationAgent` 和 `ReinforcementAgent` 等基础类。
`Game/Core/utilities.py`	定义了一些有用的类。
`Agents/Runtime/feature_extractors.py`	用于抽取特征的函数。
`Agents/Runtime/dqn_agent.py`	Q7 的神经网络训练和测试流程。

Part 1. 课前热身

Q0. 上手体验格子世界

下载并解压 zip 文件，进入文件夹目录后，在终端命令行输入以下命令来体验格子世界，通过方向键进行控制：

```
python gridworld.py -m
```



你可以看到两个出口，分别为奖励 1 和 -1。蓝点表示智能体，当你按向上键的时候，智能体实际向北移动的概率为 80%。

你可以控制模拟的多个参数。运行以下命令可以查看完整的选项列表：

```
python gridworld.py -h
```

以下命令可以让默认的智能体随机移动：

```
python gridworld.py -g MazeGrid
```

你可以看到随机智能体在网格中四处碰撞，直到偶然找到出口，这并不是智能体最理想的表现。

注意：在网格世界的 MDP 中，智能体首先必须进入一个预终止状态（在 GUI 中以双框显示），然后执行特殊的退出动作，才会使得整个回合真正结束（进入称为 `TERMINAL_STATE` 的终端状态，在 GUI 中未显示）。如果你手动运行一个回合，由于折扣率的影响（使用 `-d` 参数更改，默认为 0.9），总回报可能低于预期。

查看控制台输出（或使用 `-t` 显示纯文本输出），你可以看到智能体每次状态转换的详细信息（可以使用 `-q` 关闭此信息）。

与勇者斗恶龙游戏一样，位置由 `(x, y)` 笛卡尔坐标表示，数组的索引为 `[x][y]`，其中 `'north'` 表示 y 增加的方向。默认情况下，大多数状态转换的奖励为零，但你可以使用 `-r` 来更改这一设定。

课前热身提交内容

请阅读相关代码，并在 elearning 上提交以下问题的答案（一行代码或多行代码）。

以下问题均针对 Gridworld：给定 MDP 对象 `mdp` 和状态 `state`，请通过 MDP 提供的接口作答。

问题 0.1：给定一个状态 `state`，如何获取所有可能的动作（`action`）？

问题 0.2：如何判断一个状态 `state` 是否是终止状态？

问题 0.3：给定一个状态-动作对 `(state, action)`，怎么得到可能的后续状态及对应的概率？请以二元组的形式保存在一个列表中。